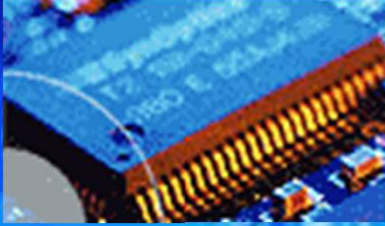
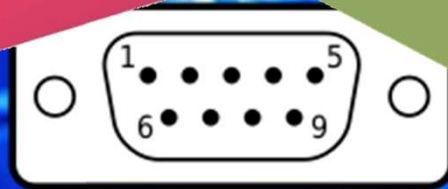


EEE3096S



**Embedded
Systems II**



Embedded Communications 3/3

Embedded Systems II

L11

Dr Simon Winberg



Electrical Engineering
University of Cape Town

ES Communications Training Plan

- Comms 1:
 - Intro: Serial vs Parallal, Synch/Asynch
- Comms 2:
 - I2C & SPI
 - UART
 - RS232 & RS485
 - PWMs & Peripherals (not used yet in Prac2)
- Comms 3:
 - Timers
 - Briefly, some other interface protocols
 - {Brief Introduction to Wireless Communications – if time permits}

The SPI Protocol



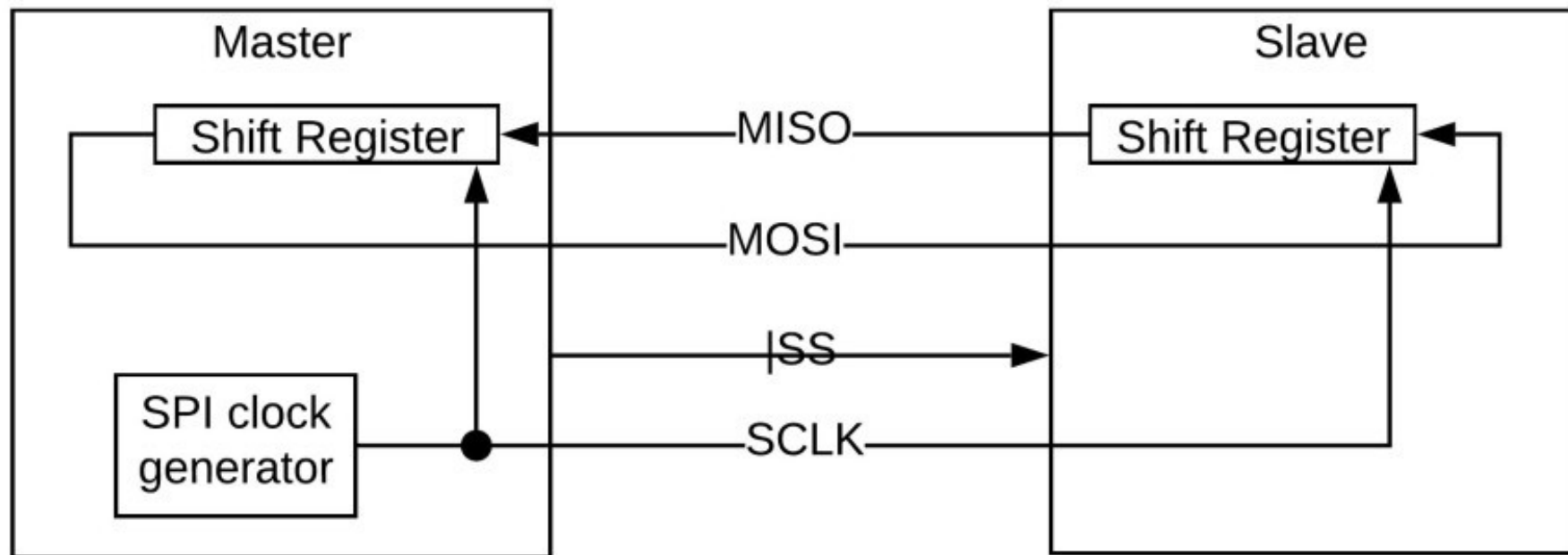
SPI Overview

SPI = Serial Peripheral Interface

- Communication protocol developed by Motorola (Freescale) in 1979
- **Full duplex** (both ends can transmit simultaneously)
- **Serial** (bits sent sequentially),
- **Synchronous** (uses a shared clock)
- **Faster than I2C**, but slightly more complex and requires more lines.
- **No Start/Stop** commands, or addressing, instead a **dedicated slave select line** is used
- Supports multiple slave devices on the same bus with a **single master**
- Also used on PCBs and between nearby chips (only works over short distances).
e.g. ADC, Temperature sensor, RTC, SD card, EEPROM

SPI Overview

SPI Interface Lines



Lines required:

- **SCLK**: Clock generated by master
- **MOSI**: Master Out Slave In
- **MISO**: Master In Slave Out
- **!SS**: Slave Select, active low (hence !)

// SS sometimes called CS=Chip Select

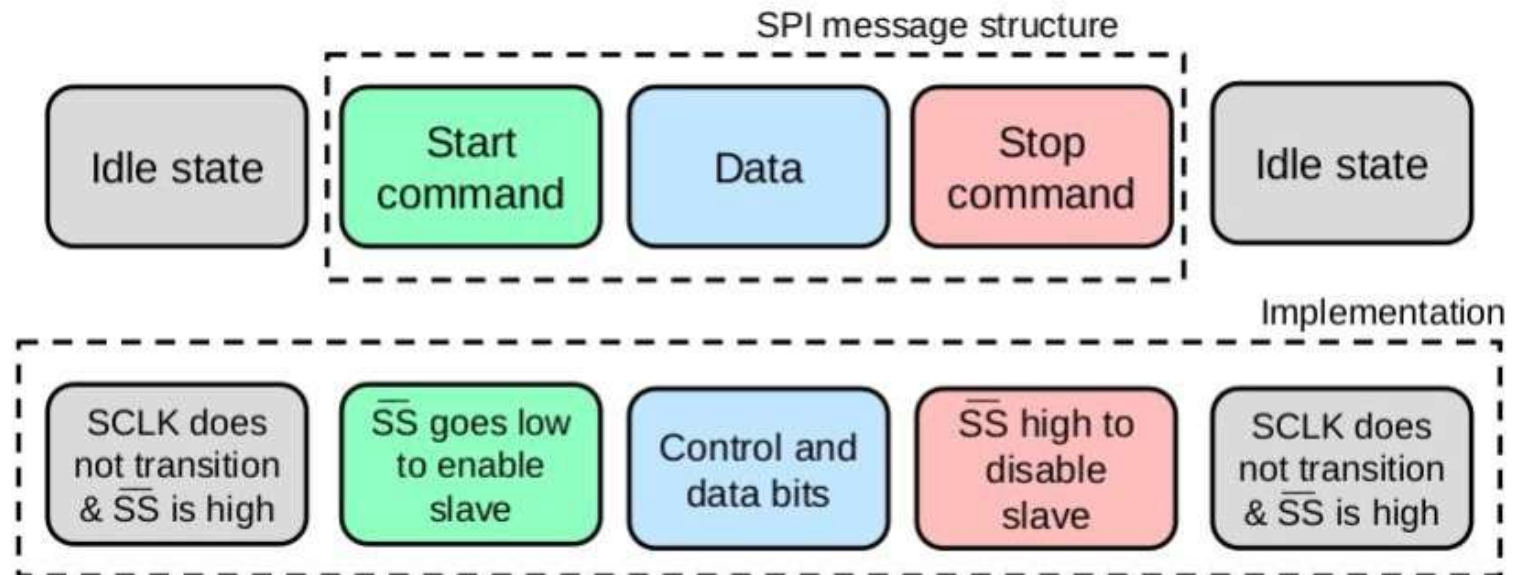
Can use | or ! to indicate active low

Operation:

Master-generated CLK moves data onto the data transfer lines (MOSI, MISO) and from/into the end point shift registers simultaneously.

SPI Protocol

Top: state of operation Bottom: what happens on signal lines

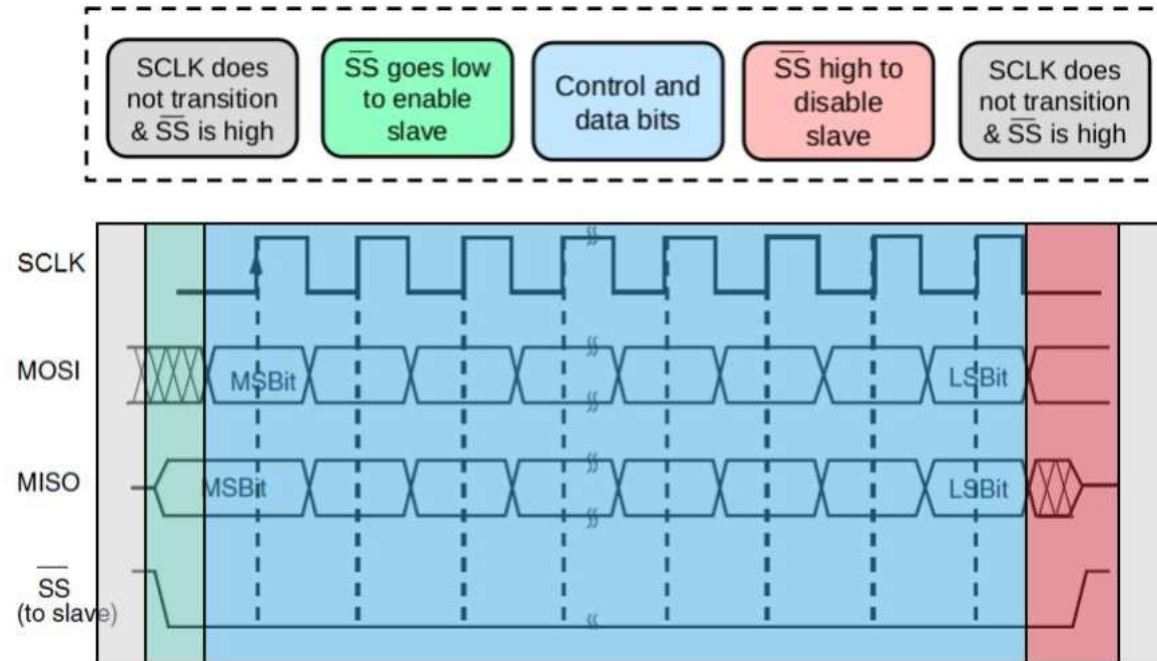


Three pretty simple parts...

1. **Idle state:** SCLK not switching, !SS HIGH
2. **!SS pulled LOW:** Starts data transfer from the 1st clk edge (rising or falling depending on mode)
3. **!SS going HIGH:** Disables any slaves and stops data movement

SPI Protocol

Top: state of operation *Bottom:* what happens on signal lines

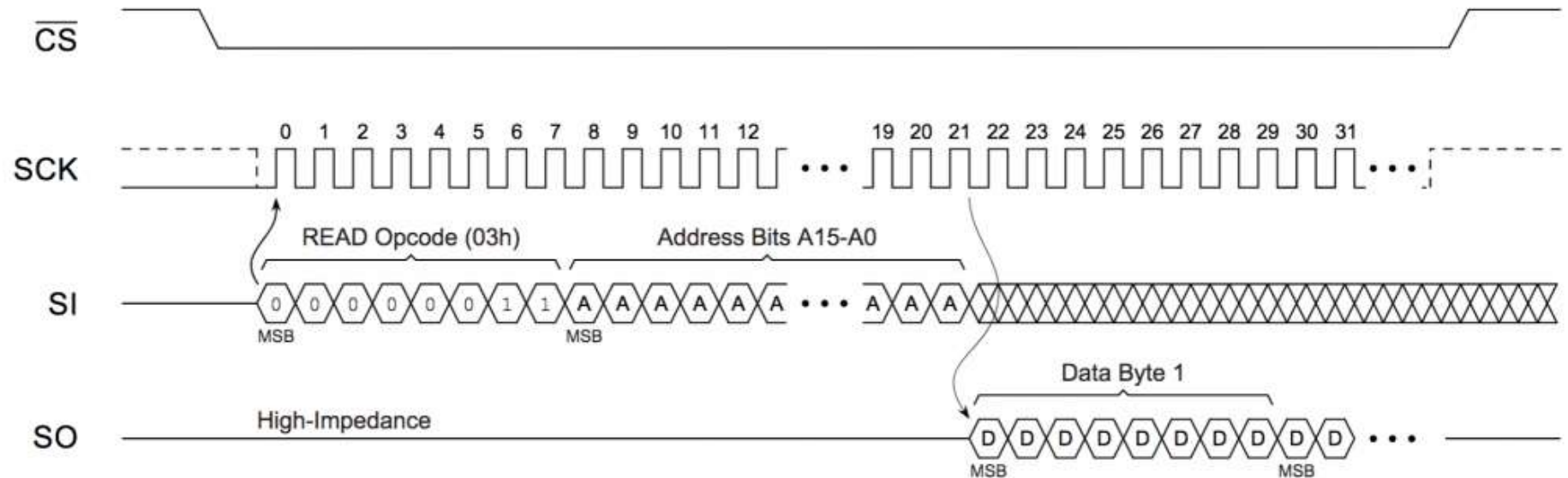


Three pretty simple parts...

1. **Idle state:** SCLK not switching, !SS **HIGH**
2. **!SS pulled LOW:** Starts data transfer from the 1st clk edge (rising or falling depending on mode)
3. **!SS going HIGH:** Disables any slaves and stops data movement

Example transmission in spi

Figure 7-1. Read Waveform



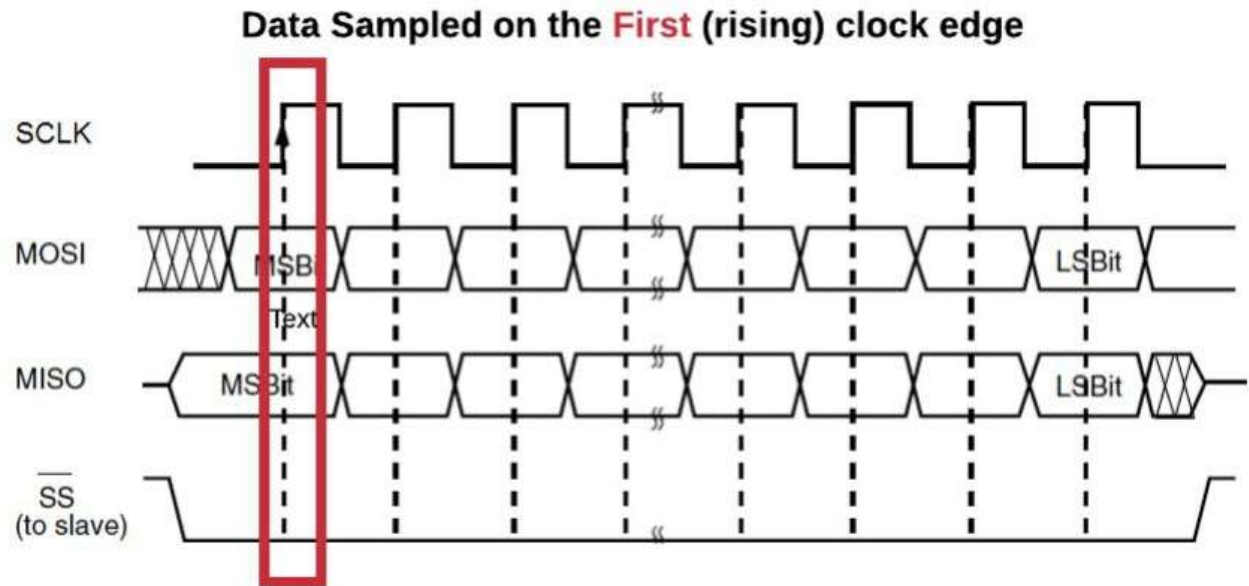
SPI Protocol – Clock Phase (CPHA) Settings

SPI Settings ...

Rising clock edge



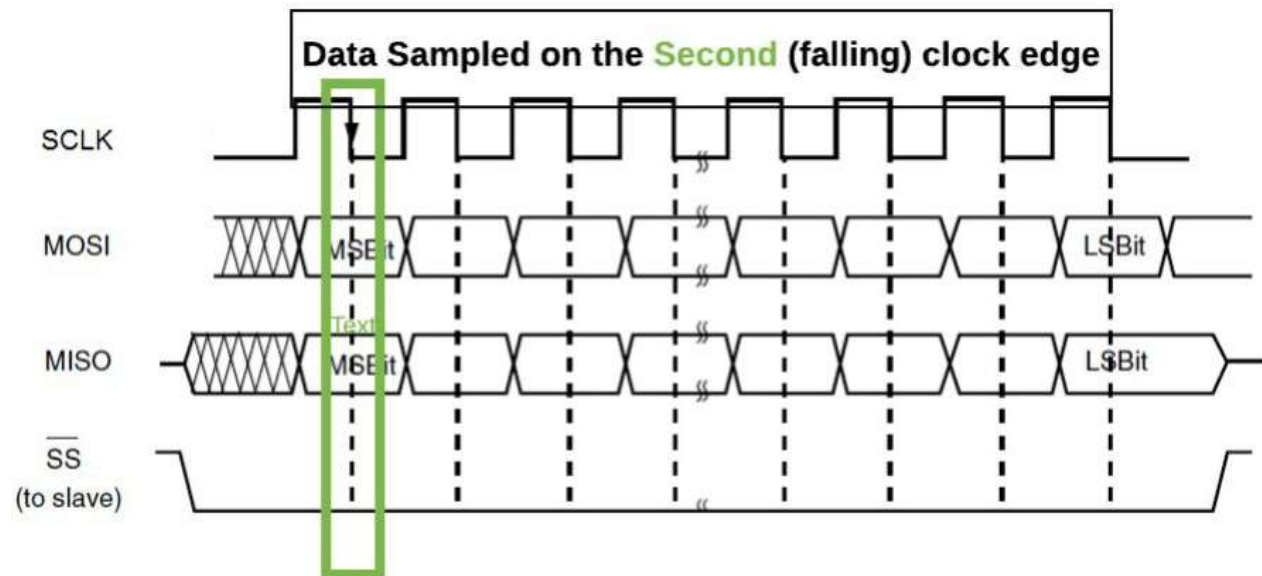
Data sampled on the
first clock edge (**rising**)



Falling clock edge



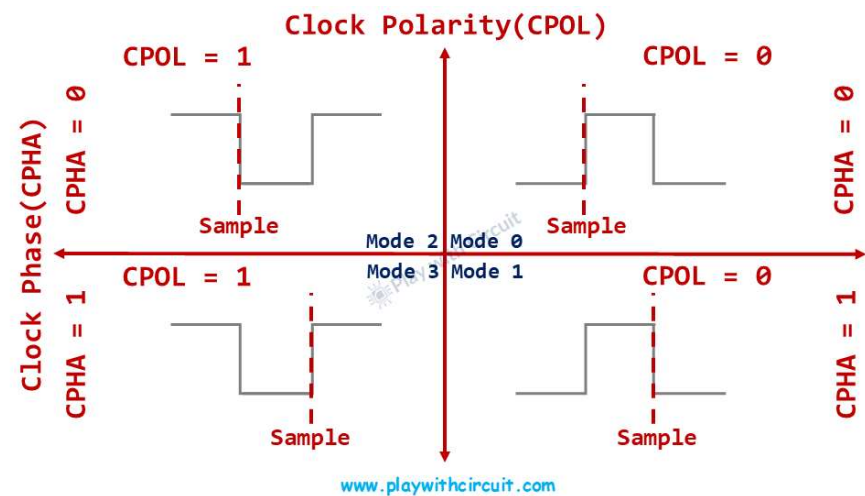
Data sampled on the
second clock edge (**falling**)



SPI Modes

- CPHA – clock phase, this determines at which edge the data is sample
= 1 (rising edge)
= 0 (falling edge)

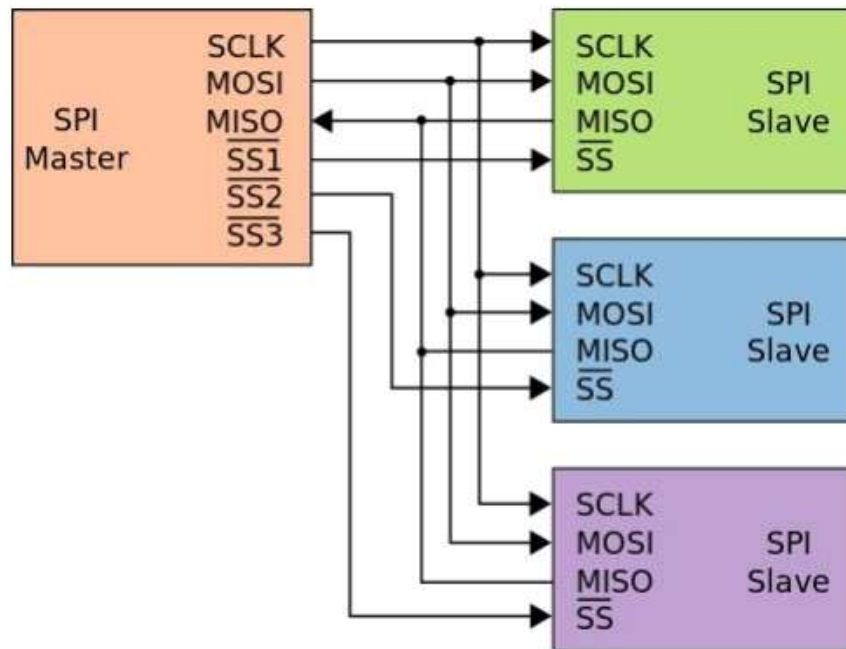
CPOL – clock polarity, determines the idle state of a clock line.
=0 (Low)
=1 (High)



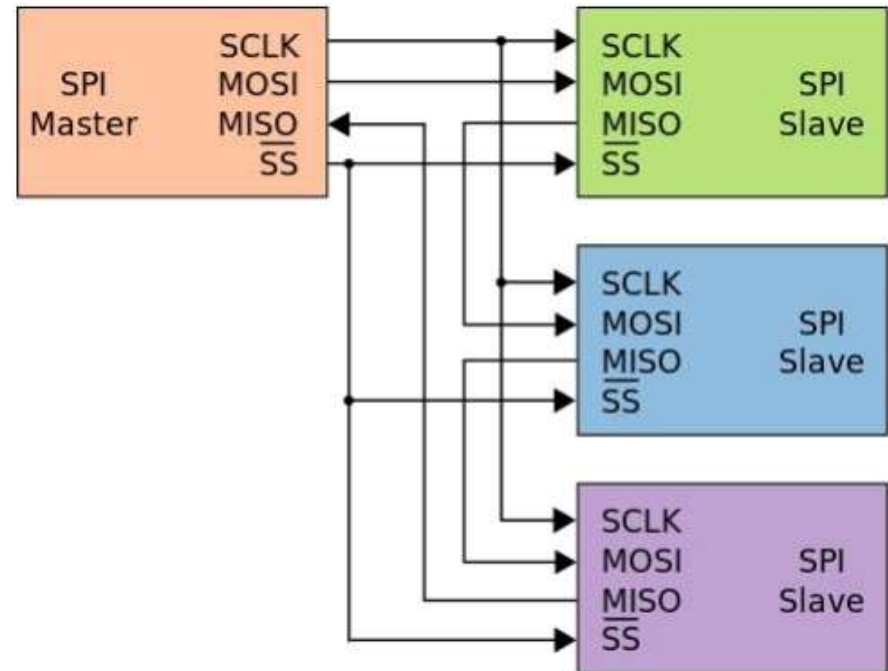
Falling edge – towards the default

SPI: Multiple Slaves

Approaches to set up SPI connections for one Master with multiple slave devices.



A typical SPI bus configuration with dedicated slave select lines: e.g. Master and three independent slaves. In this configuration each new slaves requires an additional dedicated !SS line for individual control



Daisy-chained SPI bus: master and cooperative slaves. Each slave much forward on the MOSI input out its MISO output on the next SCLK. (can allow for more devices connected up with shorter cables, but catch is higher latency)

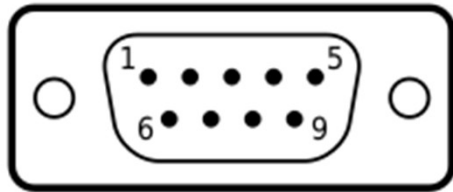
Might appear
in a test!

SPI in context

- Key Disadvantage:
 - Only 1 master possible
 - If independent access to many slaves is required the number of !SS lines quickly grows
- Key Advantages:
 - No overhead of addressing and start/stop/ACK
 - Full duplex

UART

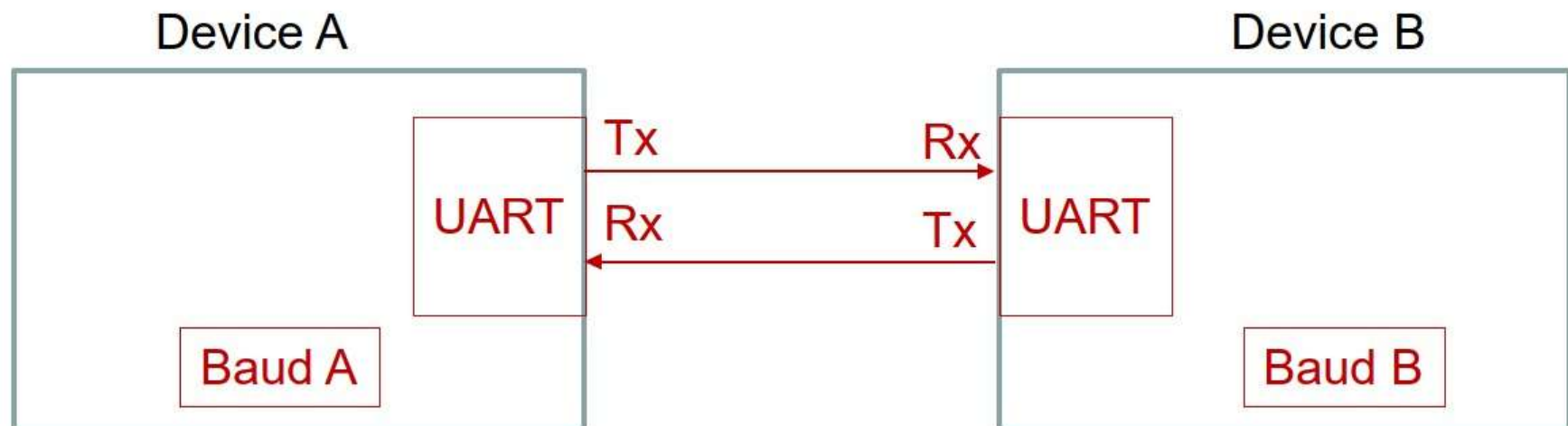
Universal Asynchronous Receiver Transmitter & RS232 Protocol Standard



What is meant by UART?

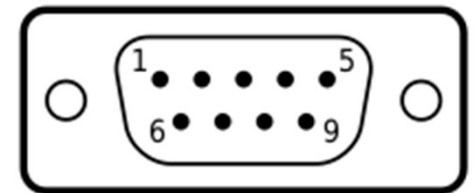
- The Universal Asynchronous Receiver Transmitter (**UART**) refers to the generic **hardware interface** Tx/Rx control module, not to a protocol! A UART could be an underlying driver that implements a protocol.

It is simply this type of setup ...



UART & RS232

- RS-232 is an asynchronous serial communication **interface standard**
- This standard was introduced in 1960 for serial communication transmission of data.
 - It formally defines signals connecting between a **DTE** (data terminal equipment) e.g. computer, and a **DCE** (data circuit-terminating equipment), e.g. modem.
 - Has 25 and 9 pin connector options (25 pin ver. seldom used, few benefits over 9 pin)



UART & RS232

- How does UART and RS232 relate?!

Nowadays the approach is to control the RS232 lines via a UART module, commonly built in to a microcontroller or an onboard component. ... and

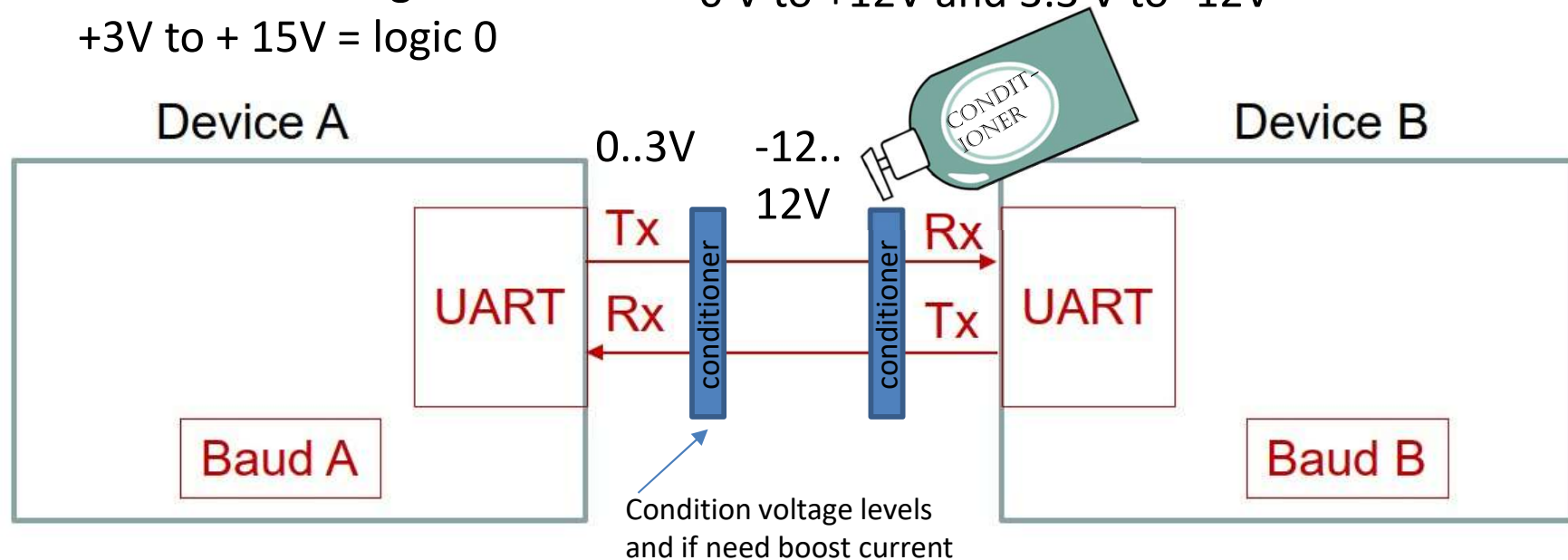
Adding essentially some voltage conditioning circuitry.

RS232 voltage levels:

-3V to -15V = logic 1 and
+3V to +15V = logic 0

Use a MAX232 to convert:

0 V to +12V and 3.3 V to -12V



RS232 Standard Configuration

- Speeds of.
 - 1.2kbps, 2.4kbps, 4.8kbps, 9.6kbps, 14.4kbps, 19.2kbps, 115.2kbps *
- RS232 was originally intended more for bit rates lower than 20kbps, but was kept going and still useable way beyond that.

The max RS232 speed is around 1Mbps

* here k = 1000

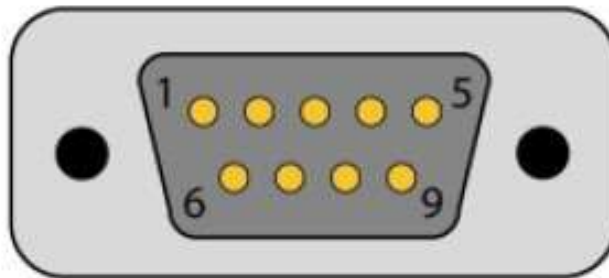
Max limit for UART? No explicit limit, left to developer to decide.

UART and the RS-232

- DB9 connector coupling

Pinouts

DB9M Connector



DB9-F
female connector



DB9-M
male connector



Pin#	Signal
1	DCD
2	RX
3	TX
4	DTR
5	GND
6	DSR
7	RTS
8	CTS
9	RI

USB to RS232 Adapter

- USB is a (seriously) complicated protocol, although very useful and well planned.
- Here's how to avoid hassles of having to figure out how to choose a use a standard USB protocol ...

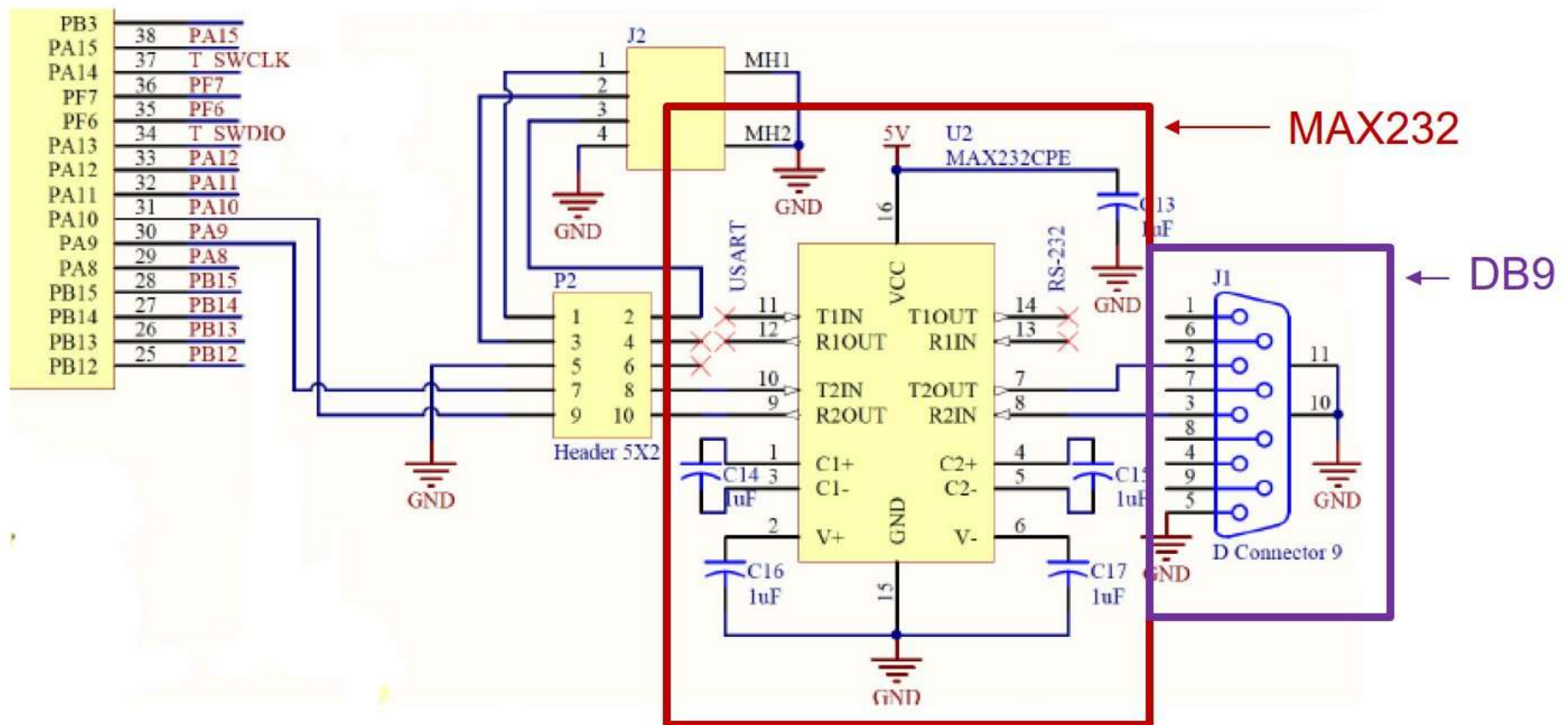


Just give your PC a RS232 port via connecting a converter via USB... easier approach.

Of course the downside to this is that your comms rate is going to be limited (don't expect much, if even all that close, to 1Mbps)

RS232 on STM32 UART → MAX232 → DM9

Illustration of connections needed to provide a RS232 port on the STM32F0:



Portion of schematic for STM32F0 development board.

Source: Embedded Systems I course notes

RS-485

- What is RS-485?
 - It's kind of an enhanced version of RS-232
 - It's also an electrical interface
 - Also usually uses UART hardware interface to drive the data exchange
 - RS-485 was developed to overcome the limitations of RS-232

RS-232 Limitations

- Supports only one transmitter and one receiver on a communication bus (you might have noticed that after our I2C and daisy-chained SPI cases)
- Allows data transfer for distances *less than* 15m. Signal lines (tx and rx) are referenced w.r.t ground and performance degrades quickly when there is noise present.
- Allows rather limited speed of data transfer (especially for lengths beyond about a ~1m)

RS-485 differences to RS-232

How is RS-485 different to RS-232?

1. RS-485 uses a tri-state differential line driver to provide one of the following three states:
 - Logic low: $\pm 1.5\text{ V}$ to $\pm 6\text{ V}$
 - Logic high $\pm 1.5\text{ V}$ to $\pm 6\text{ V}$
 - High impedance: Not connected to bus

Big advantage of this adjustment:

This allows multiple devices to be connected to the same bus. When a device is not transmitting, the transmit line is put to 'High impedance'.

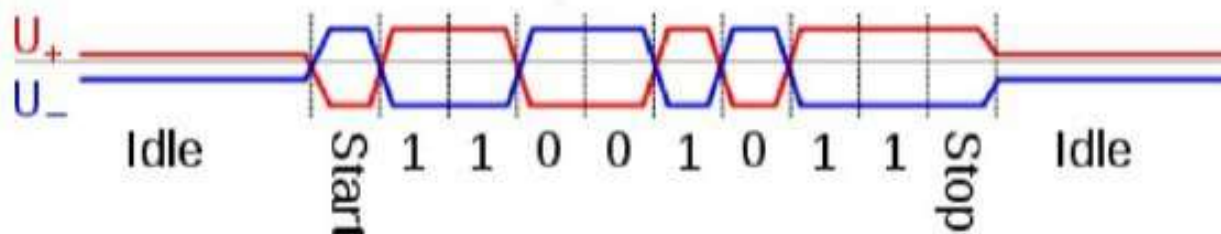
RS-485 differences to RS-232

How is RS-485 different to RS-232?

2. RS-485 uses **differential signaling on two lines** rather than a single-ended with voltage referenced to ground.

‘Differential signaling’ means that two lines are used, where one line has the negative voltage or the complementary signal of the other line. The receiver amplifies the difference between the two signals.

This makes RS-485 more noise immune than RS-232



Voltages on the U₊ and the U₋ signal lines during transmission of one byte (0xD3 or b11001011)

RS-485 differences to RS-232

How is RS-485 different to RS-232?

3. Recommended to use a **twisted pair cable** to transmit RS-485 signals. A twisted pair minimizes the gap between the two wires, which minimizes noise picked up by both the wires and immunity against interference.



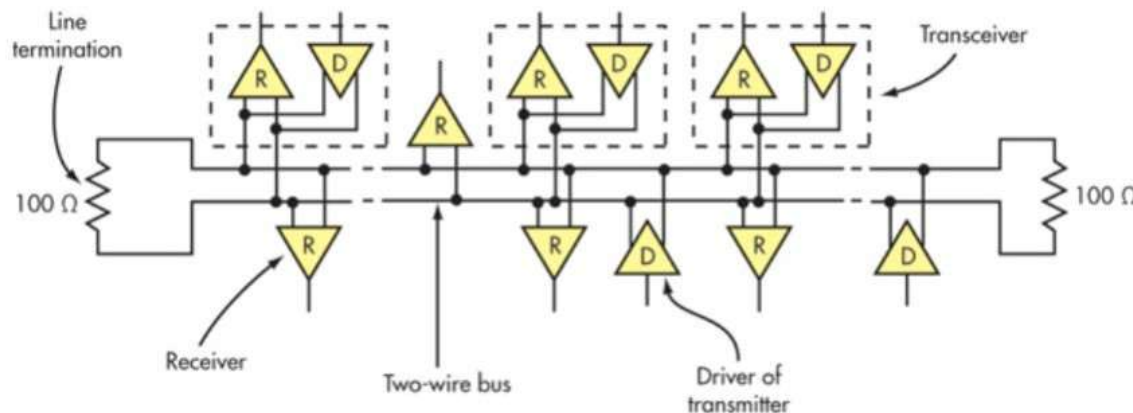
Components of a half duplex RS-485 network, where a twisted pair cable is recommended to improve noise immunity

RS-485 differences to RS-232

How is RS-485 different to RS-232?

4. Supports up to 32 transmitters and 32 receivers on one communication bus

- All receivers are fully connected to the communication bus
- Line drivers are disconnected or put in a 'high impedance' state when they are not transmitting
- The communication bus line is terminated in a load matching resistance, to minimise signal reflections from the end of the line s:



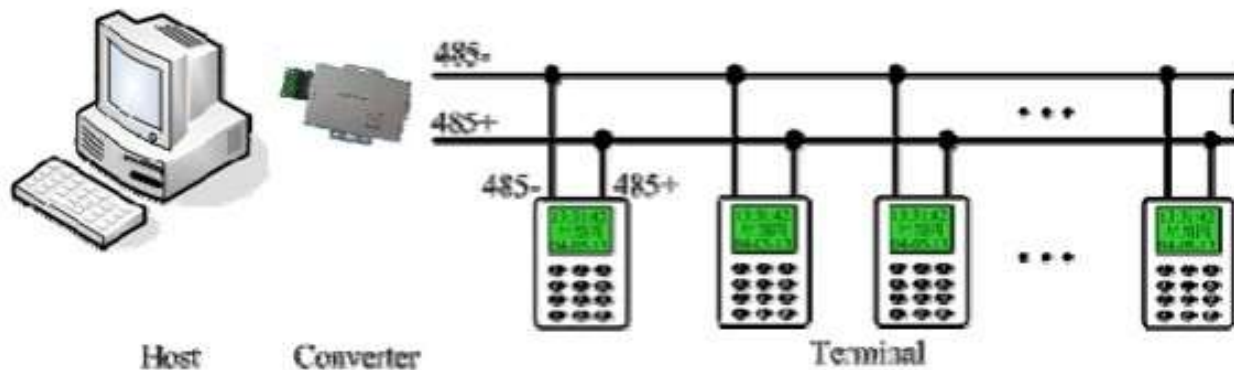
A RS-485 network showing many receivers(R) and transmitters / drivers (D) connected to the same communication bus

RS-485 versus RS232 Summary

Characteristics of RS232 and RS485		
Parameter	RS232	RS485
Cabling	Single-ended	Differential
Numbers of devices	1 transmitter 1 receiver	32 transmitters 32 receivers
Mode of operation	Simplex or full duplex	Simplex or half duplex
Maximum cable length	50 feet	4000 feet
Maximum data rate	20 kbits/s	10 Mbits/s
Signaling	unbalanced	balanced
Typical logic levels	$\pm 5 \sim \pm 15V$	$\pm 1.5 \sim \pm 6V$
Minimum receiver input impedance	$3 \sim 7k\Omega$	$12k\Omega$
Receiver sensitivity	$\pm 3V$	$\pm 200mA$

RS-485 Applications

- Where is RS-485 used?
 - Commercial aircraft cabins
 - Building automation
 - Connecting electrical meters
 - Industrial control systems
 - Security electronics



Example of multiple devices connected to the same RS-485 network. A RS-485 to RS232 convertor may be used to interface to a host

Selection of References

- [1] Understanding the I2C Bus
(Texas Instruments application note)
<https://www.ti.com/lit/an/slva704/slva704.pdf>
- [2] Serial Peripheral Interface (wikipedia)
https://en.wikipedia.org/wiki/Serial_Peripheral_Interface

Suggested further reading/understanding:

[STM32F0 MINI TUTORIAL – USING THE I2C \(in C\)](#)

[MikroC SPI Library](#) (shows some STM code examples but these are more direct H/W register access, gives a view of what happens within the STM32CubeIDE HAL layer)

Suggested Reading*

- Resolving I2C Address Conflicts

<https://embeddedartistry.com/blog/2021/08/02/resolving-i2c-address-conflicts/>

(this is to consolidate I2C understanding and also awareness of the advantage and also drawback of I2C addressing aspect).

* This is of course more I2C and Prac2 related, although taking those principles further from what we would likely use in this course or the project.